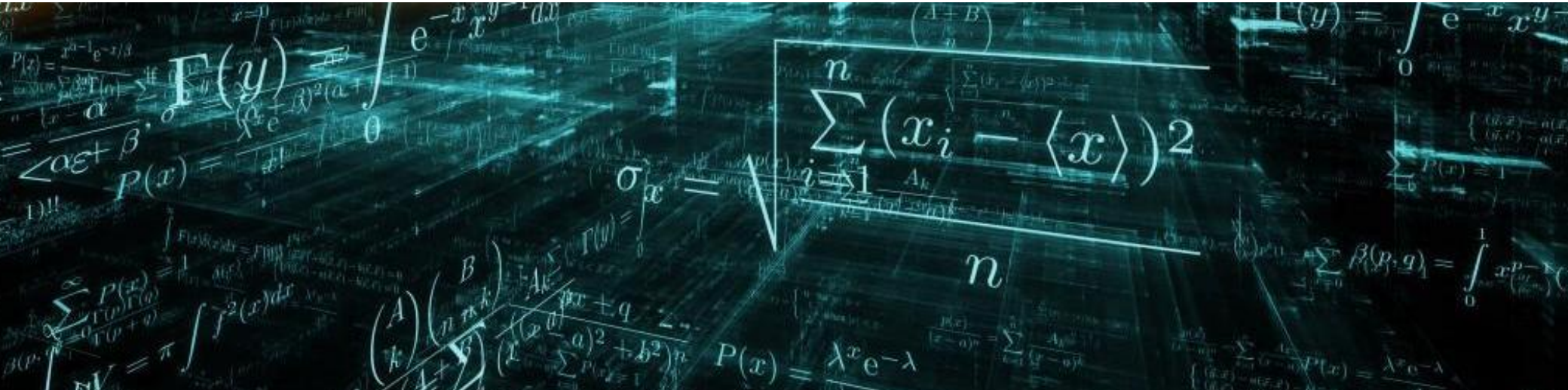
Multimedia Retrieval

Chapter 7: Retrieval Augmented Generation

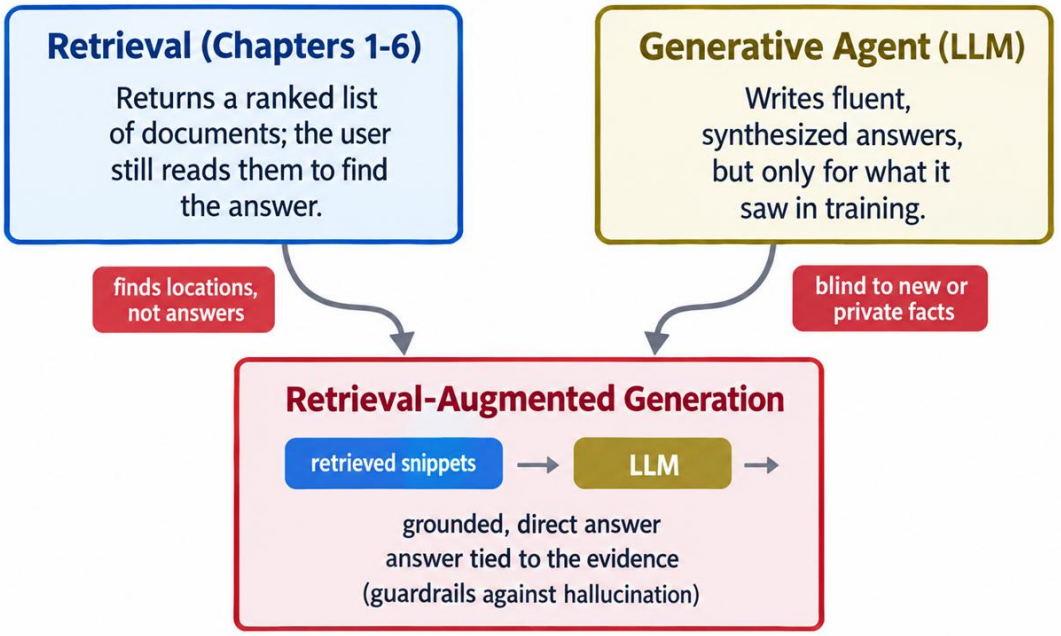
Dr. Roger Weber, roger.weber@gmail.com

7.1 The RAG Pipeline
7.2 Chunking Strategies
7.3 Result Generation
7.4 Agentic RAG
7.5 Evaluating RAG



From a ranked list to a direct answer

- **Retrieval** returns a ranked list; the user extracts the answer
  - right for "compare gaming PCs"; poor for "who won the 2025 Nobel Prize in Physics?"
- **Generative agent** answers fluently, but only from training
  - cannot answer about this year, private policies, unseen documents
- **RAG** combines both: retrieved snippets feed the model, which writes a **grounded** answer
  - retrieval spares agent from reading the collection
  - the agent spares user from reading the list
- New hard problem: **hallucination**, a confident answer without the facts

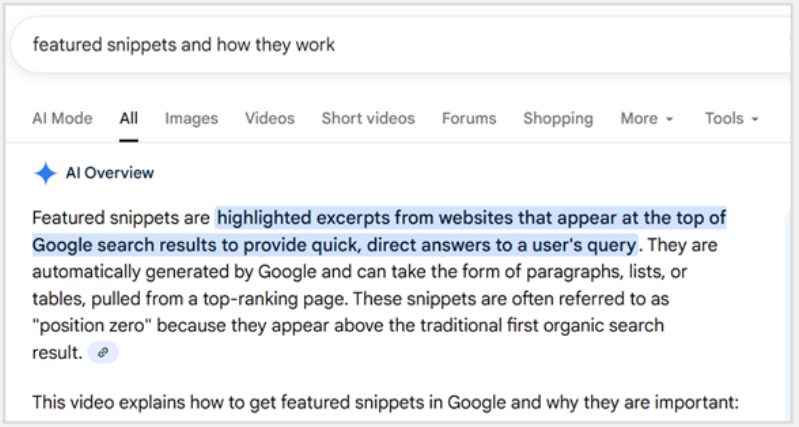


Why retrieve at all: three limits of parametric knowledge

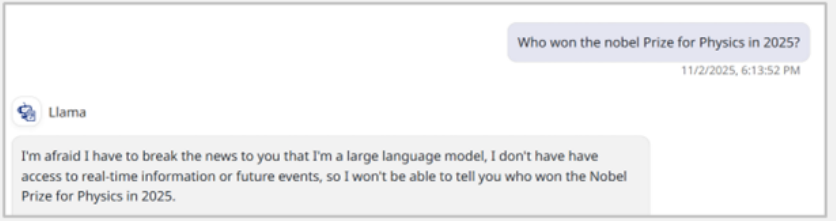
- **Time**: weights frozen at training; this year's laureates are absent
- **Scope**: no model read your internal policies or private collection
- **Trust**: fluent text whether or not it has the facts; hallucination

- **Fine-tuning**: costly, slow, stale once the facts change
- **RAG**: keep the model fixed, change **what it reads**
  - answer pulled toward evidence
  - passages can be **cited** and checked

AI Overview, Google (2024)



Generation-only is not always successful

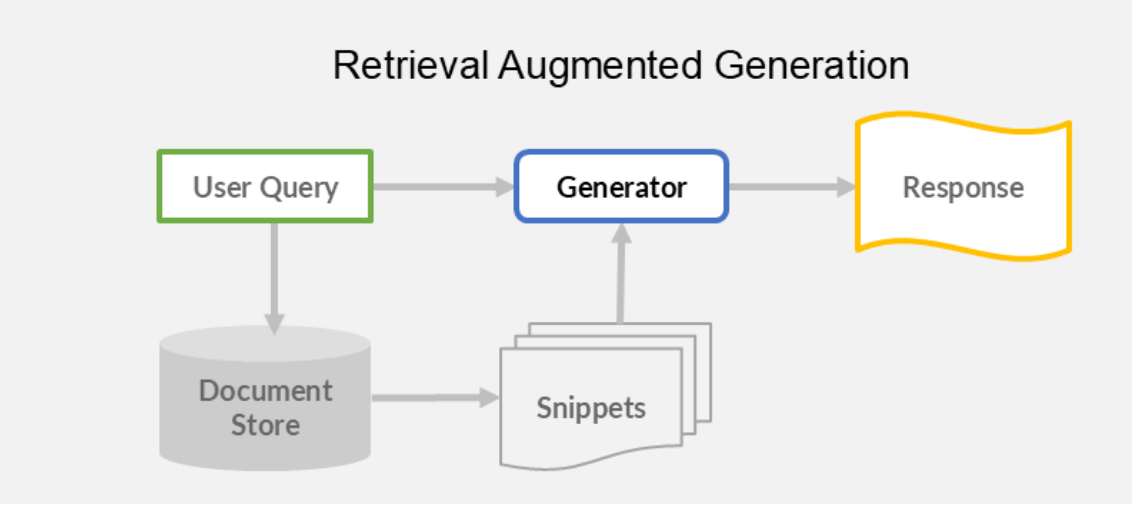


The minimal loop, and why two steps are not enough

- **Minimal RAG**: the query goes two ways
  - retrieves snippets from a document store
  - prompts the generator, which combines query and snippets

- **Early RAG** = retrieve, then generate; inherits each retriever's weakness
  - **BM25** misses "the 2025 laureate **was awarded**" for "who **won**": no shared tokens
  - **dense** bridges wording but blurs detail: "who" (a person) dissolves in the embedding; returns pages about the prize

- Fix: **chain stages**, each repairing the previous one's weakness



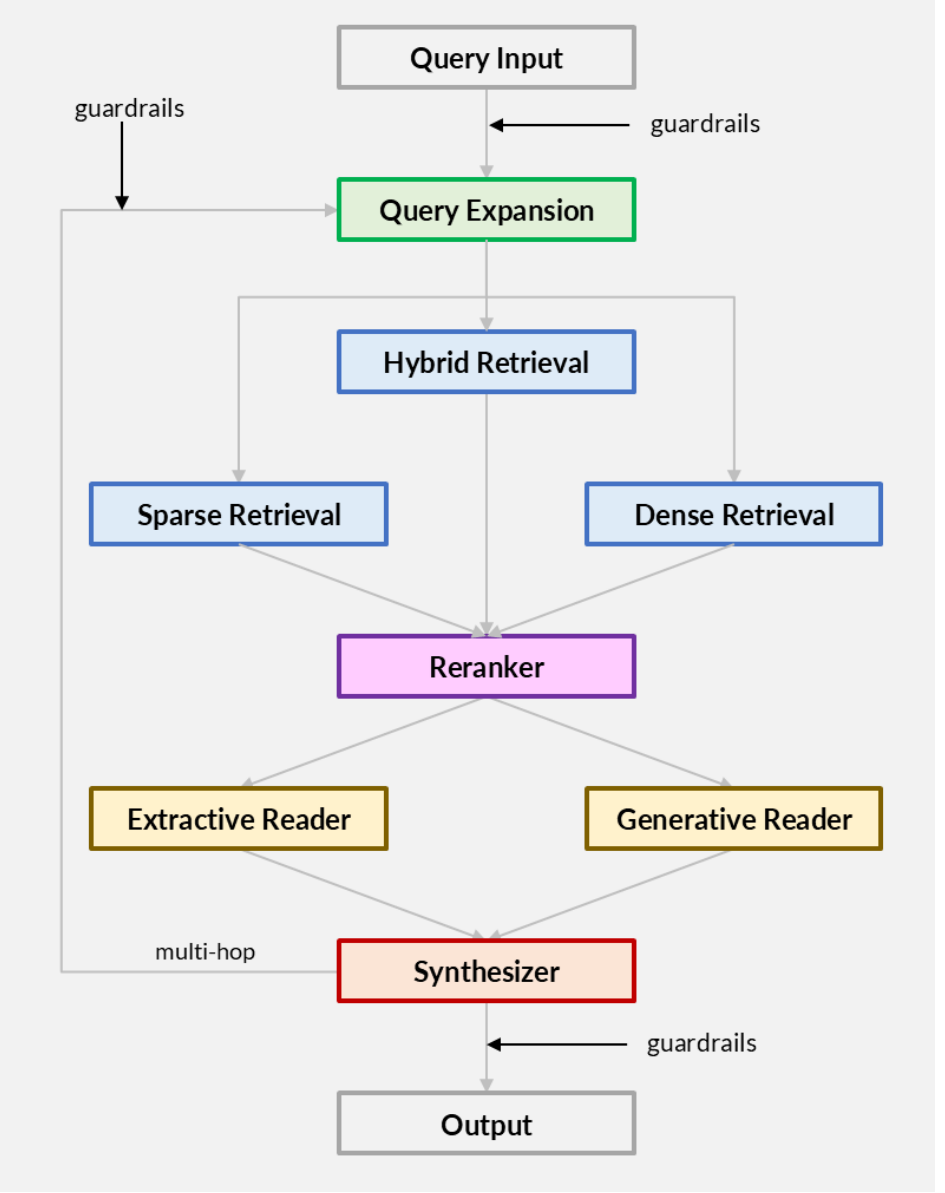
The advanced RAG pipeline

- Running example: "Who won the Nobel Prize in Physics in 2025?"

- **Query expansion**: spelling, synonyms, rewrites, sub-questions
  - LLM patterns: HyDE (search for a plausible answer), multi-query, step-back

- **Sparse + dense retrieval**, merged by **hybrid** fusion (RRF)
- **Reranking**: cross-encoder cuts a few hundred candidates to a handful; sees that "who" wants a person

- **Result generation**: reader, generator, synthesizer
- **Guardrails** at input and output; **multi-hop** loop back to the start



What RAG reuses, what it adds

- **Mostly familiar**: chapters 3 to 6 supply expansion, retrieval, reranking
- **Two new stages**, because the consumer changed from a human to a model
  - **two-level chunking**: small chunks to match, parents to answer from
  - **result generation**: span, direct answer, or cross-document summary
- **Modular and tunable**: every stage optional and swappable
  - factual lookup: skip dense and reranking
  - hard domain question: full chain
  - route easy queries down a short path
- Latency relaxes: a streamed answer over seconds feels natural

Stage	From	Trade-off
Query expansion	ch3 query understanding	better recall, one more step
Sparse (BM25)	ch4 inverted index	fast, shallow on meaning
Dense retrieval	ch5 embeddings, ch6 ANN	captures meaning, more compute
Reranking	ch5 cross-encoder	highest precision, most expensive per candidate
Two-level chunking	new	sharp matching and enough context
Result generation	new	answer instead of list; hallucination risk

Building the pipeline in Haystack

- **Frameworks** assemble stages from components
  - Haystack, LangChain, LlamaIndex
  - Haystack: typed components, a graph connects them
- **Indexing pipeline**: splitter, embedder, writer fill the store
- **Query pipeline**: BM25 + dense in parallel, RRF, cross-encoder, generator
- Each component = one stage; **swap** in one line
- **Limitation**: the graph is fixed when built
  - one pipeline per intent; new intent = new code

```python
rag = Pipeline()
rag.add_component("bm25", InMemoryBM25Retriever(document_store=store))
rag.add_component("embedder", SentenceTransformersTextEmbedder())
rag.add_component("dense",
                  InMemoryEmbeddingRetriever(document_store=store))
rag.add_component("join",
                  DocumentJoiner(join_mode="reciprocal_rank_fusion"))
rag.add_component("rerank",
                  SentenceTransformersSimilarityRanker(top_k=5))
rag.add_component("prompt", ChatPromptBuilder(template=template))
rag.add_component("llm", AmazonBedrockChatGenerator(model=MODEL))

rag.connect("embedder.embedding", "dense.query_embedding")
rag.connect("bm25", "join")
rag.connect("dense", "join")
rag.connect("join", "rerank")
rag.connect("rerank", "prompt.documents")
rag.connect("prompt", "llm")

q = "Who won the 2025 Nobel Prize in Physics?"
result = rag.run({"bm25": {"query": q}, "embedder": {"text": q},
                  "rerank": {"query": q}, "prompt": {"query": q}})
```

▶ **Demo**: ch07-02-rag-pipeline.ipynb

Chunking: the size trade-off

- Embedding a whole book as one vector: close to **nothing in particular**
- Chunking is older than RAG: show the best passage, matched terms close together
  - RAG revived it: early embedders lost accuracy beyond a few hundred tokens; small context windows
- Today a deliberate **trade-off**
  - **larger** chunks: more context for the generator
  - **smaller** chunks: sharper embeddings, one clear meaning
- Strongest resolution: **two levels** (hierarchical chunking)

	Smaller chunks	Larger chunks
Embedding	sharp, one meaning	blurred average
Generator context	may lack context	room to reason
Suits	factual lookups	summaries, cross-topic questions

Sweet spot: roughly 512 to 1024 tokens; a starting point, not a rule

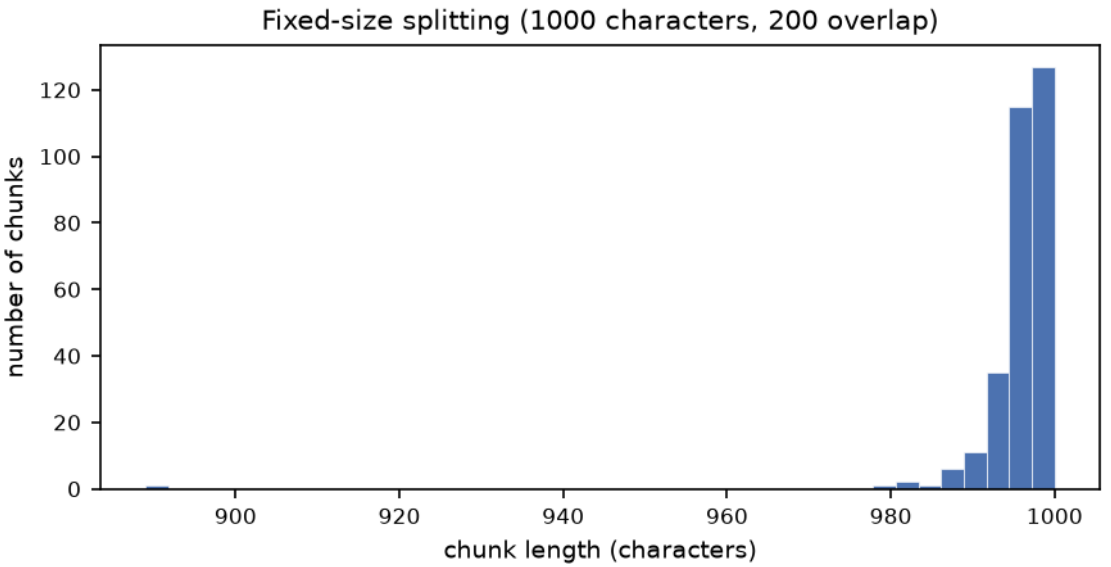
Fixed-size splitting

- Ignore structure: group words up to a **target size**
- **Overlap** consecutive chunks, so a split sentence appears whole in one of them

```python
from langchain_text_splitters import CharacterTextSplitter

text_splitter = CharacterTextSplitter(
    separator=" ",          # break on word boundaries
    chunk_size=1000,        # target characters per chunk
    chunk_overlap=200,      # characters shared between neighbours
    add_start_index=True,   # record each chunk's offset in the source
)
chunks = text_splitter.split_text(study_in_scarlet)
```

- On A Study in Scarlet (1000 chars, 200 overlap)
  - **299 chunks**, all between 889 and 1000 characters
  - variation only from stopping at the last whole word



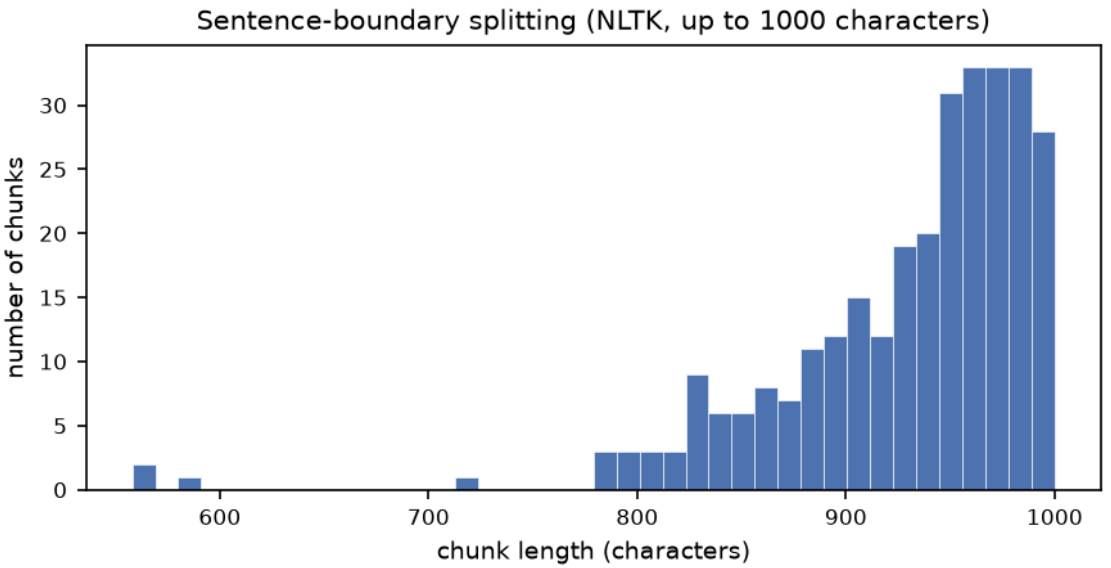
- Easy and fast; enough for rough locality
- **Ignores meaning**: a chunk can mix the end of one idea with the start of another; embeddings blend concepts

Sentence boundaries

- Smallest unit = **sentence**: add whole sentences up to the limit
  - never breaks mid-sentence; reads naturally
- Boundary detection is not "split on every period"
  - "Dr.", quoted dialogue; Punkt and spaCy segmenters (ch3)
- On A Study in Scarlet: **299 chunks**, mean ~930 characters
  - wider spread, down to ~560: a chunk closes when the next sentence would overflow
- Still blind to **topic shifts**: two unrelated adjacent sentences share a chunk

```
# or SpacyTextSplitter
from langchain_text_splitters import NLTKTextSplitter

text_splitter = NLTKTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = text_splitter.split_text(study_in_scarlet)
```

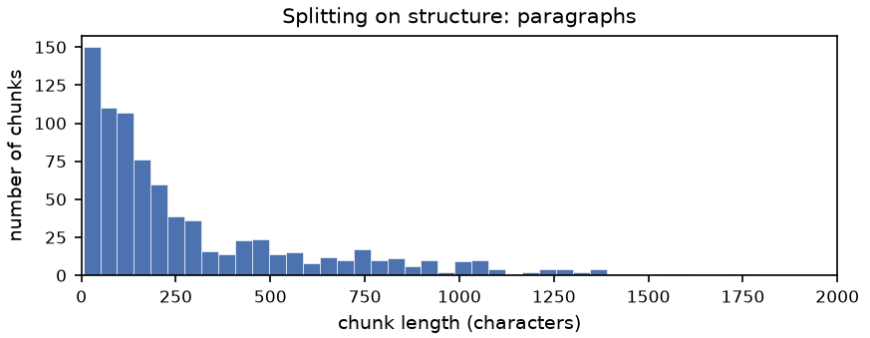


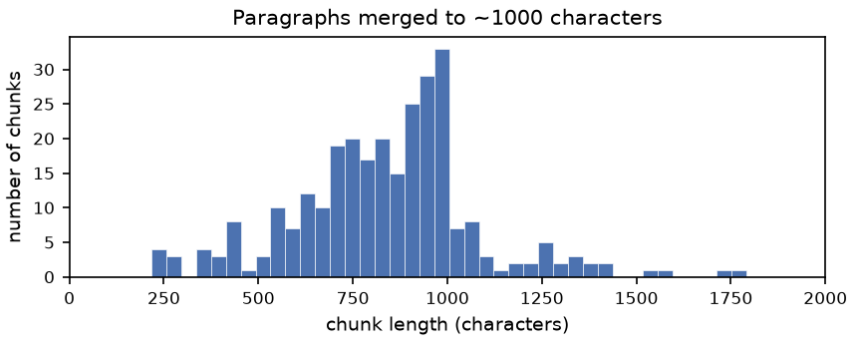
Splitting on structure

- Use the **author's units**: chapters, sections, paragraphs
  - Gutenberg: 4 newlines = chapter, 2 = paragraph; HTML/Markdown: headings, paragraph tags
- Coherent, but **sizes far from uniform**
  - by chapter: **19** pieces, mostly 10,000 to 23,000 chars; too big
  - by paragraph: **816** pieces; dialogue makes over half < 200 chars
- Fix: **merge** small adjacent paragraphs
  - typical chunk ~850 chars; a few long ones > 1500
  - overlap: last sentence of the previous + first of the next chunk

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    # four newlines = chapter, two = paragraph
    separators=["\n\n\n\n", "\n\n"],
    chunk_size=2000,  # large cap: keep each structural unit whole,
    chunk_overlap=0,  # only splitting a unit that exceeds the cap
    add_start_index=True,
)
chunks = text_splitter.split_text(study_in_scarlet)
```





Hierarchical chunking: two levels at once

- One chunk size cannot serve **both** retrieval and generation
  - so store more than one granularity; the arrangement most suited to RAG
- **Split coarsely, then finely, keep both**
  - coarse: headings or sections
  - fine: sentence, fixed-token, or semantic splitter within each
- Each level has a **job**
  - small chunks: accurate embeddings, drive **retrieval**
  - parent section: handed to the **generator** as context
  - stored metadata links each sub chunk to its parent

```python
from langchain_text_splitters import (
    MarkdownHeaderTextSplitter,
    RecursiveCharacterTextSplitter,
)

header_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "H1"), ("##", "H2"), ("###", "H3")]
)
sections = header_splitter.split_text(long_markdown_text)

token_splitter = RecursiveCharacterTextSplitter(
    chunk_size=512, chunk_overlap=50)
final_chunks = []
for sec in sections:
    subs = token_splitter.split_text(sec.page_content)
    for idx, sub in enumerate(subs):
        final_chunks.append({
            "content": sub,
            "metadata": {**sec.metadata, "sub_chunk": idx},
        })
```

Which chunking strategy is best?

- **No universal winner**: depends on documents, questions, reader

- Chunking happens **before** any query
  - answer straddles a boundary: recall suffers
  - relevant sentence in unrelated text: dilute context
  - a size histogram cannot show this

- **Measure the task**, vary only the chunking
  - **answer recall@k**, **context precision**
  - end-to-end answer quality

- Smell test: similarity **within** vs **across** chunks

$$answer\ recall@k = \frac{|\{q \in Q : answer(q) \in top_k(q)\}|}{|Q|}$$

answer(q) ∈ top_k(q): at least one of the top k chunks contains the answer to q

Unlike document recall (ch2): counts one hit per question, not the share of all relevant documents found

▶ **Demo**: ch07-01-chunking-strategies.ipynb

Result generation: reader, generator, synthesizer

- Turns top passages into an **answer**
- Trades **faithfulness** for **flexibility**
  - reader: safe but rigid
  - synthesizer: powerful, hardest to keep honest
- **Choose per query** (classifier or agent)
  - context: small, verbatim vs broad, diverse
  - complexity: simple fact vs open-ended
  - intent: exact quote vs explanation
  - **risk**: legal, medical favour the reader

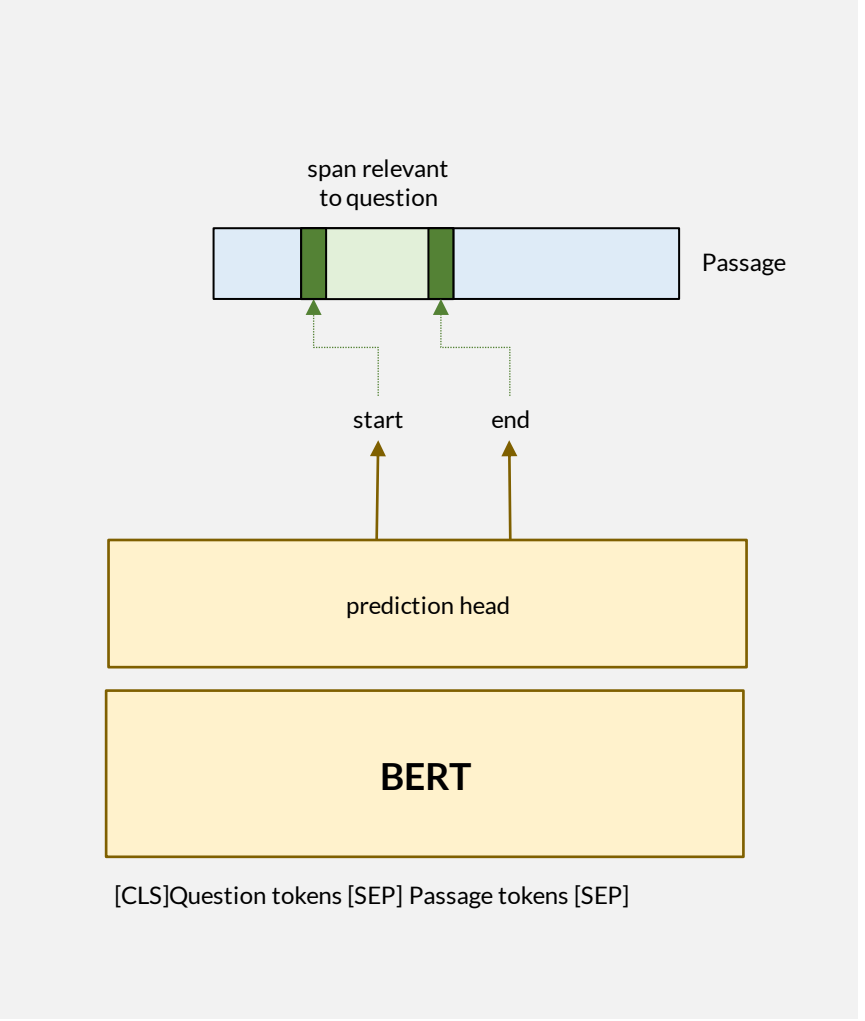
Form	How it answers	Best for	Main risk
Reader	marks and returns the answer span verbatim from one passage	exact facts, short passages, high stakes	cannot combine or rephrase; fails if no single span holds the answer
Generator	writes a new, direct answer from several passages with an LLM	open-ended questions, fluent answers across sources	hallucination; must be grounded by prompt and guardrails
Synthesizer	loops retrieval and reasoning over several documents, then summarizes	deep research, evidence across sources	latency and complexity; drift across hops

The reader: extractive span prediction

- **Reader** marks the span that answers; generates nothing
  - mechanism behind a highlighted search result

- **BERT** extractive QA
  - input: [CLS] question [SEP] passage
  - every token sees question and passage
  - head scores each passage token as **start** and **end**
  - answer = span with the highest start + end score

- Fine-tuned on **SQuAD** (gold start and end positions)



The reader in code

- Encode question and passage **together**
- Read off the span between the highest-scoring start and end tokens
  - offset mapping: token positions back to characters
- Returns the **exact span**; start and end peak sharply
- Strength = ceiling: only text that is **literally present**
  - cannot combine two half-answers or rephrase
  - why Featured Snippets needed no LLM: a reader only points, it does not write

```python
import torch
from transformers import AutoTokenizer, AutoModelForQuestionAnswering

name = "distilbert-base-uncased-distilled-squad"
tokenizer = AutoTokenizer.from_pretrained(name)
model = AutoModelForQuestionAnswering.from_pretrained(name)

question = "Who won the Nobel Prize in Physics in 2025?"
context = (
    "John Clarke, Michel Devoret, and John Martinis won the 2025 "
    "Nobel Prize in Physics for their discovery of macroscopic "
    "quantum tunnelling."
)

enc = tokenizer(question, context, return_offsets_mapping=True,
                return_tensors="pt")
offsets = enc.pop("offset_mapping")[0]
with torch.no_grad():
    output = model(**enc)

start = output.start_logits.argmax()   # highest-scoring start token
end = output.end_logits.argmax()       # highest-scoring end token
answer = context[offsets[start][0]: offsets[end][1]]
print(answer)
# John Clarke, Michel Devoret, and John Martinis
```

The generator: the prompt is the lever

- **Generator** writes token by token, merges passages
  - that freedom is where **hallucination** enters
- A grounded prompt
  - **task**; passages = **only source of truth**
  - no prior knowledge; restricted **scope**
  - **output format**; tone, audience
  - answer **missing**: say "I don't know"
- **Order and phrasing** matter; meta-prompting refines the prompt
- **Fluent is not grounded**: check claim support

```
PROMPT = """
You answer questions about {domain} using only the
passages below. Cite the passages you rely on by id,
like [1]. If the answer is not in them, say you don't
know rather than guessing.

<passages>
{passages}
</passages>

<question>
{query}
</question>

Answer in {format}, and base every claim on the
passages above.
"""
```

Task and scope first, tagged context, question last, grounding rule repeated at the end

The synthesizer: multi-hop evidence

- **Synthesizer** = generator + **iteration**
  - retrieve, read, find the gap, follow-up query
  - an early, hard-wired agent loop

- "Where do the 2025 physics laureates work today?"
  - **names no laureate**: one pass finds prize lists
  - hop 1: extract the names; hop 2: one query per name

- Pays off when the question does not name its subject; costs latency

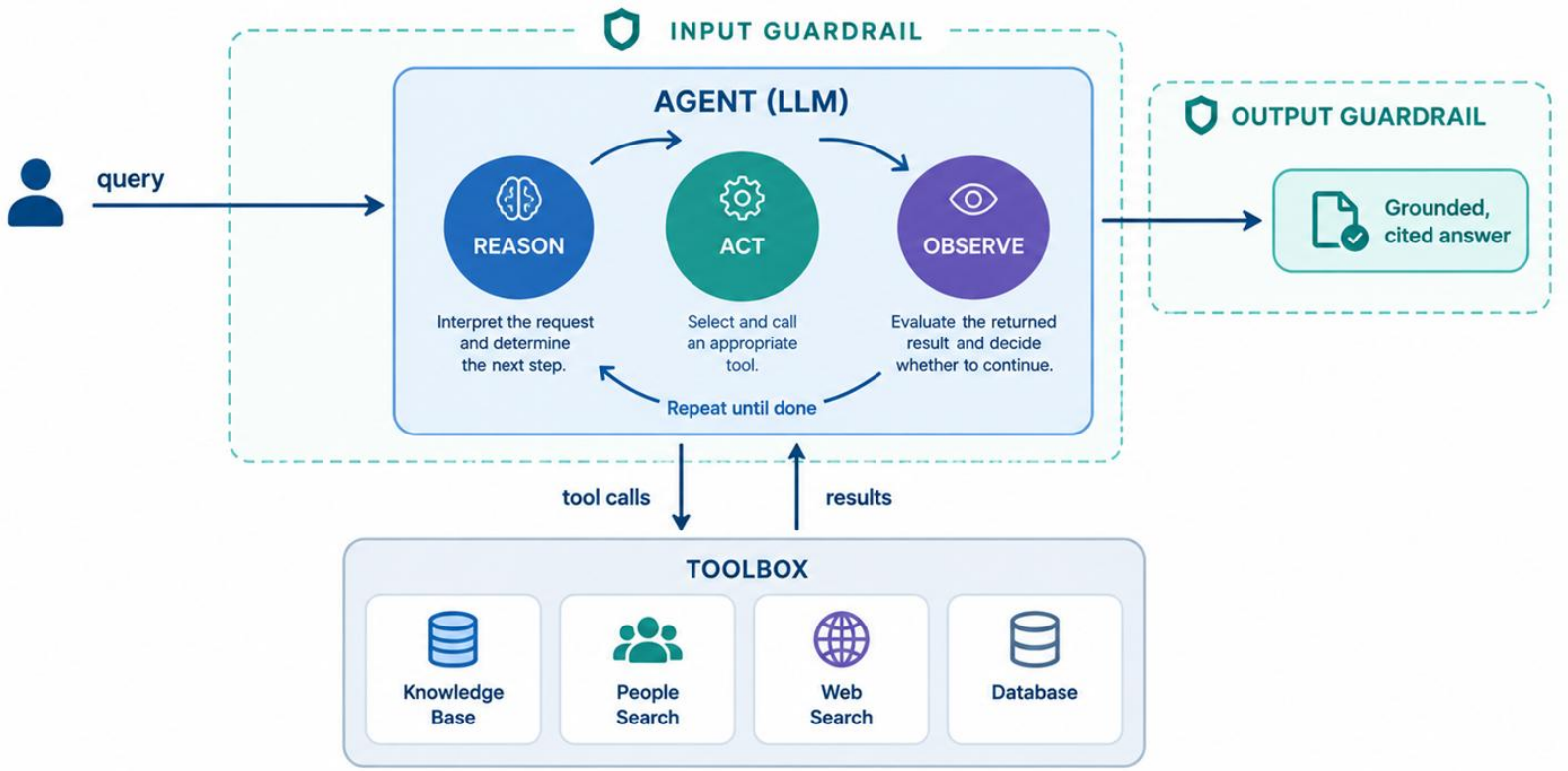
Approach	Passage budget	Institutions reached
Single pass	9	Santa Barbara
Two hops	9	Berkeley, Yale, Santa Barbara, Saclay, Google

Same budget, Nobel knowledge base of the chapter demos

▶ **Demo**: ch07-03-result-generation.ipynb

From fixed pipelines to the agent loop

- **Fixed pipeline** = one intent
  - every change is a code change
  - "winners, and where each works?" = 1 lookup + 1 per winner: not anticipated, not expressible
- **Agent loop**: reason, act (tool call), observe, repeat (ReAct)
  - stop: enough evidence, or the iteration budget
- **Guardrails** at input (scope, safety) and output (grounding), at every hop



Walking the loop: the 2025 Physics Nobel

- "Who won the 2025 Nobel Prize in Physics, and which universities are they affiliated with?"
- Tools: **Wikipedia** knowledge base, **people-search** (JSON), **web search** as fallback
- Shift: stop writing the sequence, **declare capabilities**
- The loop takes over
  - **intent detection**: the first thought, no classifier
  - **query transformation**: a well-formed query per tool
  - **multi-hop expansion** no one coded
  - vague question: ask back, or infer from the conversation

Step	Agent	Tool call	Observation
1. Intent (reason)	current facts, two parts: winners and affiliations	none	
2. Query transformation (act)	well-formed query, not the raw sentence	wikipedia_search("2025 Nobel Prize in Physics")	Clarke, Devoret, Martinis
3. Search expansion (reason, act)	affiliations missing, loop	people_search, one call per name, one step	{"name": "John Clarke", "affiliation": "UC Berkeley"}; no record for Martinis
3b. Fallback (Martinis)	try another way	other spelling, knowledge base, web_search as last resort	
4. Ad-hoc flow	winners-then-affiliations assembled from the question		
5. Final answer	evidence complete, stop	none	grounded, cited answer

Implementing the loop: declaring the tools

- The loop is short; the **tool declaration** makes it work
  - name, description, parameters (JSON Schema)
  - the **description** is what the model reads to pick a tool
- Declaring tools = telling the agent **what it may do**
  - web_search marked as last resort in its description
- **System prompt** sets the grounding rule for the whole run

```python
def schema(arg):
    """JSON Schema for a tool with one required string argument."""
    return {"type": "object", "properties": {arg: {"type": "string"}},
            "required": [arg]}

TOOLS = [
    {"name": "wikipedia_search",
     "description": "Search a Wikipedia knowledge base about the "
        "Nobel Prizes and return the text of the 3 best matching "
        "passages with their source.",
     "input_schema": schema("query")},
    {"name": "people_search",
     "description": "Look up a person in the people directory. "
        "Returns JSON with name, affiliation, and position, or an "
        "error if there is no record.",
     "input_schema": schema("name")},
    {"name": "web_search",
     "description": "Search the web. Returns titles and short "
        "snippets. Use only when the other tools return nothing.",
     "input_schema": schema("query")},
]

SYSTEM = ("Answer with the tools provided. Base every statement on "
          "tool results and name the source of each fact. If the "
          "tools do not give the answer, say so.")
```

Implementing the loop: reason, act, observe

- One pass = one **reason-act-observe** cycle
  - llm: conversation + tools to Claude
  - call_tool runs it; tool_result = observation
  - reply without tool call: **done**
  - several calls per step (one per laureate)
- **Message list** = working memory
- **max_steps** = stop condition
- Recurring flow: a **skill** in the system prompt
  - Haystack agent, LangGraph package the loop

```python
def run_agent(question, max_steps=8):
    messages = [{"role": "user", "content": question}]
    for _ in range(max_steps):                  # iteration budget
        reply = llm(messages, tools=TOOLS, system=SYSTEM)   # reason
        messages.append(reply.message)
        if not reply.tool_calls:           # no further action: done
            return reply.text
        results = [tool_result(call.id,
                               call_tool(call.name, call.arguments))
                   for call in reply.tool_calls]   # act
        messages.append({"role": "user",
                         "content": results})      # observe
    return "I could not complete the answer within the step budget."
```

▶ **Demo**: ch07-04-agentic-rag.ipynb

Evaluating RAG: more choices, more dimensions

- Classical: how good is one ranking function? RAG: which **combination** serves users best?
- Beyond precision and recall
  - **answer quality**, **latency**, **cost**
- **Price of an answer**: must be clearly better, not merely different
  - latency as **p95**: the slow cases users notice
  - cost **per answered question**: input + output tokens + tool charges
- **Agents**: the trajectory matters too
  - iterations, tools called, right call?, kept searching after enough evidence?
  - same answer in 2 calls beats 10

Choice	Example options	Mainly affects
Document processing	chunking strategy, chunk size, overlap	retrieval recall, index size
Retrieval	embedding model, BM25 weight, top-k	recall, precision, retrieval latency
Reranking	reranker model, candidates reranked	precision, latency
Generation	generator model, prompt, context size	answer quality, token cost
Agent control	tools available, iteration limit	answer quality, number of tool calls, total latency

Retrieval still sets the ceiling

- An answer is never better than its **evidence**

- **Context recall**: was everything needed retrieved?
  - a missing fact cannot be recovered; at best admit the gap, at worst invent it
  - needs a labelled reference

- **Context precision**: how much of the context is relevant?
  - noise competes for attention; closely related but wrong evidence is the most dangerous
  - often judged without a reference

- Same trade-off as classical: more retrieved, more recall, more noise

$$context\ recall = \frac{|required\ evidence\ retrieved|}{|required\ evidence|}$$

$$context\ precision = \frac{|relevant\ chunks\ retrieved|}{|chunks\ retrieved|}$$

System	Retrieved	Required found	Context recall	Context precision
Fixed pipeline, top 5	winners, 1 affiliation, 2 Chemistry, 1 ceremony	2 of 4	2/4 = 0.5	2/5 = 0.4
Agent	winners, then one lookup per laureate	4 of 4	1	1

Required: one passage naming the laureates + one affiliation each

Measuring answer quality: faithfulness

- Four questions, overlapping, not interchangeable
  - **correctness**: factually right? vs a reference
  - **faithfulness**: every claim supported by the context? grounding
  - **relevance**: addresses the question asked?
  - **completeness**: all required facts?
- Faithfulness is the metric **specific to RAG**
- **Faithful is not correct**
  - a true claim not in the context lowers it
  - a fully faithful answer is wrong if the sources are
  - measure correctness separately

$$faithfulness = \frac{|\{claims\ in\ the\ answer\ supported\ by\ the\ context\}|}{|\{claims\ in\ the\ answer\}|}$$

Claim in the answer	Supported by context?
The 2025 Physics prize went to Clarke, Devoret, and Martinis	yes
They shared 11 million Swedish kronor	no (true, but from memory)
faithfulness	1/2 = 0.5

Comparing two configurations

- Practical question: is change **B** better than system **A**?
  - new chunking, a reranker, an agent loop
- **Controlled experiment**
  - fixed benchmark: questions, reference answers, required evidence (ch2 benchmark design)
  - same questions; change **only one** component
  - record context, answer, latency, tokens, tool calls
- judge_* = evaluators such as RAGAS
- Tooling: RAGAS, DeepEval (metrics, regression tests); Phoenix, LangSmith (tracing)

```python
def evaluate(system, benchmark):
    rows = []
    # item: question, reference answer, required evidence
    for item in benchmark:
        start = time.perf_counter()
        # run: answer, retrieved, tool_calls, tokens
        run = system.answer(item.question)
        # stop the clock before evaluating
        latency = time.perf_counter() - start
        rows.append({
            "context_recall": recall(run.retrieved, item.evidence),
            "context_precision": precision(run.retrieved,
                                           item.evidence),
            "correct": judge_correct(run.answer, item.reference),
            "faithfulness": judge_faithful(run.answer, run.retrieved),
            "latency_s": latency,
            "tokens": run.tokens,
            "tool_calls": len(run.tool_calls),
        })
    return rows

results_a = evaluate(system_a, benchmark)
results_b = evaluate(system_b, benchmark)
```

Is B really better? The paired sign test

- 50 questions: A 76%, B 88%; luck?

- **1. Disagreements only**: 10, B wins 8
- **2. No difference**: fair coin, binomial
- **3. At least as extreme**: 8:2 + 9:1 + 10:0 = 0.055 (even 5:5 is only 0.25)
- **4. Both directions**: p ≈ 0.11

- p > 0.05: **not conclusive**
  - 100 questions, 16 of 20: p ≈ 0.012
- Fix **latency and cost** limits beforehand

	B correct	B wrong
A correct	36	2
A wrong	8	4

$$P(B\ wins\ k\ of\ 10) = \frac{\binom{10}{k}}{2^{10}}$$

$$p = 2 \cdot \frac{\binom{10}{8} + \binom{10}{9} + \binom{10}{10}}{2^{10}} = 2 \cdot \frac{45 + 10 + 1}{1024} \approx 0.11$$

▶ **Demo**: ch07-05-rag-evaluation.ipynb

When not to use RAG

- RAG is **not the default**: a simpler tool avoids cost and latency
- An **agent softens the choice**: memory, database tool, or retrieval per query
  - "RAG or not" becomes "which tool for this query"
- **Ongoing costs** even where RAG fits
  - an index to build and keep fresh, retrieval latency per query
  - ceiling set by the corpus: noisy or stale sources give confidently wrong answers
- **Right** when answers need private, changing, or specific knowledge and must be **verifiable**

Situation	Better tool	Why
Knowledge stable and general (textbook definition, historical fact)	the model alone	retrieval adds a step that can fail
Corpus fits in the context window	long-context prompting	no chance to retrieve the wrong passage
Change behaviour: style, tone, format	fine-tuning	RAG changes what the model reads, not how it writes
Structured data ("orders shipped last week")	database / API query, text-to-SQL	exact answer instead of an approximation

Key takeaways

- A ranked list leaves the answer to the user; an LLM alone cannot answer what it never saw; RAG grounds a direct answer in checkable sources
- RAG reuses BM25, embeddings, vector search, reranking, and adds two parts because a model now consumes the results: two-level chunking and result generation
- Chunking balances sharp embeddings against enough context; two levels keep both; the best strategy is the one that retrieves your answers
- Reader marks a span, generator writes an answer, synthesizer summarizes across hops; faithfulness traded for flexibility; the prompt is a real lever
- Agentic RAG: one reason-act-observe loop over declared tools replaces a pipeline per intent; intent, query rewriting, multi-hop, stop within a budget; guardrails at input and output, every hop
- Evaluation: retrieval sets the ceiling; correctness, faithfulness, relevance, completeness; latency and cost must justify the answer; compare A and B with a paired test on disagreements

Further reading

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). **Retrieval-augmented generation for knowledge-intensive NLP tasks**. *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2005.11401. The paper that named RAG and introduced the retrieve-then-generate architecture this chapter builds on.
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., et al. (2020). **Dense passage retrieval for open-domain question answering**. *EMNLP 2020*. arXiv:2004.04906. DPR, the dual-encoder dense retriever behind the dense-retrieval arm of the pipeline.
- Reimers, N., & Gurevych, I. (2019). **Sentence-BERT: Sentence embeddings using Siamese BERT-networks**. *EMNLP 2019*. arXiv:1908.10084. The sentence-embedding method used to embed chunks and queries, and to measure similarity in semantic chunking.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). **BERT: Pre-training of deep bidirectional transformers for language understanding**. *NAACL-HLT 2019*. arXiv:1810.04805. The encoder behind both the reader and the cross-encoder reranker.
- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). **SQuAD: 100,000+ questions for machine comprehension of text**. *EMNLP 2016*. Dataset. The benchmark used to train and evaluate the reader of this chapter.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). **Lost in the middle: How language models use long contexts**. *Transactions of the Association for Computational Linguistics*. arXiv:2307.03172. Shows that models use information at the start and end of a long context far better than in the middle, one reason the order of a prompt matters.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). **ReAct: Synergizing reasoning and acting in language models**. *ICLR 2023*. arXiv:2210.03629. The reason-act-observe loop underlying the agentic RAG of this chapter.
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). **RAGAS: Automated evaluation of retrieval augmented generation**. arXiv:2309.15217. The evaluation framework behind the faithfulness, answer relevance, and context precision and recall metrics of this chapter.
- deepset. **Haystack documentation.** docs.haystack.deepset.ai. Reference for the components and pipelines used to build the RAG pipeline in code.
- Firecrawl (2026). **OpenClaw vs Hermes Agent: Which one should you actually run?** *Firecrawl blog*. firecrawl.dev. A practical comparison of two personal assistants that run a single agent loop for everything a user asks.